

Guía de Pruebas de Software

Proyecto PlomerIA — Grupo 14

QA/QC · Tipos de prueba · Pruebas unitarias con PyTest · TDD · Herramientas

Documento de referencia: qué pruebas aplicamos, cómo se usan, qué funciones se prueban, cómo correrlas y qué subir al repositorio. Basado en la Clase VI (Pruebas de Software, 2026) y aplicado al código real del proyecto.

1. QA vs QC — Garantía vs. Control de calidad

Son dos enfoques complementarios de la calidad del software:

	QA (Garantía)	QC (Control)
Foco	Prevención de defectos (el proceso)	Detección de defectos (el producto)
¿Cuándo?	Desde el inicio del proyecto	Cuando hay producto para probar
¿Qué hace?	Define estándares y criterios de aceptación	Ejecuta pruebas sobre el sistema
En PlomerIA	Definir reglas claras (estados de una solicitud, penalizaciones, validación de descripción)	Probar la app: registrar, solicitar, cancelar, intentar romperla

En este proyecto hacemos sobre todo QC: probamos el comportamiento del sistema ya construido (manual y automáticamente). El QA aparece cuando definimos antes las reglas del negocio que después verificamos.

2. Tipos de prueba y cuáles aplicamos

Tipo	Qué prueba	¿Lo usamos?
Unitaria	Una función o unidad aislada. Rápida. (PyTest)	Sí — funciones del backend
Integración	Que varios módulos funcionen bien juntos	Mencionada (a futuro)
Sistema / E2E	El sistema completo, flujos de punta a punta	Manual (a futuro con Playwright)
Aceptación (UAT)	El usuario valida que cumple lo que necesita	Sí — pruebas manuales
Caja negra	Sin mirar el código: entradas y salidas	Sí — manual y de API
Caja blanca	Conociendo el código: se prueban sus caminos	Sí — las unitarias

Aclaración importante: lo que ya venís haciendo (registrarte, solicitar, cancelar, y meter datos incoherentes para ver si se rompe) **ya es testing**: son **pruebas manuales exploratorias de caja negra**, y las que buscan errores a propósito se llaman **pruebas negativas**. Sirven para la parte de aceptación.

3. Formalizar las pruebas manuales (casos de prueba)

Tu testing manual, escrito como tabla de casos de prueba, ya es un entregable. Se completa la columna 'Real' al ejecutar cada caso:

#	Qué pruebo	Pasos / entrada	Resultado esperado	Real
1	Cancelar sin plomero	Solicito y cancelo antes de que acepten	Cierra sin penalización	✓
2	Descripción inválida	Escribo 'asd' como problema	Rechaza y pide describir mejor	✓
3	Rechazo total (negativa)	Los 4 plomeros rechazan	Avisa y deja volver a buscar	✓
4	Agotar reintentos	Rechazan 3 rondas más	Se cierra sola, cartel de disculpas	✓

#	Qué pruebo	Pasos / entrada	Resultado esperado	Real
5	Finalizar antes de hora	Marco finalizado antes del día	El botón está bloqueado	✓

Las filas 3 y 4 son **pruebas negativas**: intentan llevar el sistema a situaciones límite para confirmar que responde bien.

4. Pruebas unitarias con PyTest (caja blanca)

Prueban **una función aislada**: le doy una entrada y verifico (con `assert`) que devuelve lo esperado. Son ideales para las funciones 'puras' del backend (las que solo calculan y no tocan la base de datos). En PlomerIA tenemos varias:

Función (archivo)	Qué hace	Casos a probar
oficios.py (registro)	Qué oficios/especialidades están habilitados; detección por palabras clave	Que solo Plomería esté activa; que detecte 'caño/gotea'; que NO sugiera un oficio inhabilitado
_franja_de_hora()	Mapea una hora a mañana/tarde/noche	Los casos borde: 12 (mañana), 17 (tarde), 18 (noche)
_calcular_fecha_trabajo()	Convierte un turno en la fecha real del trabajo	Que el día y la hora coincidan; que no caiga en el pasado
Penalización por cancelación	Calcula estrellas según comunicación y tiempo	Con/sin comunicación, antes/después de 24 h

Cómo se escribe una prueba (ejemplo real, ya incluido en el proyecto):

```
from services import oficios

def test_solo_plomeria_habilitada():
    habilitados = oficios.oficios_habilitados()
    assert "PLOMERIA" in habilitados
    assert "ELECTRICIDAD" not in habilitados # definida pero inhabilitada

def test_detectar_plomeria():
    assert oficios.detectar_oficio("se tapó el caño y gotea") == "PLOMERIA"
```

Cada función que empieza con `test_` es un caso. Si el `assert` se cumple, PyTest lo marca en verde; si no, en rojo con la línea exacta del fallo.

Cómo definir qué probar (caja blanca):

- Mirá la función y sus **ramas** (cada `if` es un camino a cubrir).
- Probá un caso **típico** por cada rama (mañana, tarde, noche).
- Probá los **casos borde**: los límites (la hora 12, la 17), valores 0, vacío, None. Ahí se esconden la mayoría de los bugs.
- Probá un caso **negativo**: entrada inválida → que devuelva None o error controlado, no que explote.

5. Pruebas para 'romper' el código (negativas y de API)

Buscan que el sistema falle de forma **controlada** ante entradas o acciones indebidas. Dos formas:

- **Manual / interfaz**: lo que ya hacés — datos incoherentes, acciones fuera de orden. (A futuro se automatiza con Selenium/Playwright manejando el navegador solo.)
- **De API (caja negra)**: le pego a un endpoint con un caso inválido y verifico el error. Ej.: cancelar una solicitud en un estado no permitido debe devolver 400, no romperse.

Herramientas: **Postman** (manual, visual, guardás los requests en una colección) o el **TestClient** de **FastAPI** dentro de PyTest. Ejemplo de la idea:

```
# Pseudo-ejemplo de prueba de API (caja negra)
def test_cancelar_estado_invalido(client):
    r = client.patch("/api/solicitudes/999/cancelar")
    assert r.status_code in (400, 404) # error controlado, no un 500
```

6. TDD — Test Driven Development

Invierte el orden: primero el test, después el código. Ciclo:

- **RED**: escribís un test que falla (el código todavía no existe).
- **GREEN**: escribís el código mínimo para que pase.
- **REFACTOR**: limpiás el código sin romper el test.

No hace falta rehacer todo con TDD. Alcanza con **aplicarlo en una función nueva** (por ejemplo, al sumar un oficio) y contarle en la defensa.

7. Herramientas de testing

Herramienta	Para qué	En PlomerIA
PyTest	Pruebas unitarias en Python	La que usamos
Postman / Newman	Pruebas de APIs REST	Recomendada para los endpoints
Selenium / Playwright	Automatizar el navegador (E2E)	A futuro, para los flujos del front
SonarQube	Analiza el código: bugs, 'code smells', cobertura	Para mencionar (detecta cosas como los print de debug)
Appium	Testing de apps móviles	No aplica (la app es web)

8. IA en pruebas (2026)

- **Generación de casos:** herramientas como Copilot o Claude generan pruebas unitarias y cubren casos borde.
- **Testing visual con IA:** detecta diferencias en la interfaz automáticamente.
- **Análisis predictivo:** predice qué funciones tienen más probabilidad de fallar.
- **Self-healing tests:** los tests de UI se 'auto-reparan' cuando cambia la interfaz.

9. Cómo correr las pruebas y qué subir a GitHub

Estructura en el proyecto: las pruebas viven en Backend/tests/.

```
Backend/  
  tests/  
    conftest.py      # configuración (encuentra los módulos)  
    test_oficios.py  # 12 pruebas del registro de oficios  
    test_franjas.py  # 6 pruebas de la franja horaria  
    requirements-dev.txt # pytest (dependencias de prueba)
```

Para correrlas en la terminal de VS Code:

```
cd Backend  
pip install -r requirements-dev.txt  
pytest
```

La consola muestra algo como tests/test_oficios.py [100%] y al final 18 passed. Lo que falla aparece en rojo con la línea exacta.

Qué SÍ subir a GitHub (con un commit normal): la carpeta tests/ entera (conftest + archivos test_*.py) y requirements-dev.txt.

Qué NO subir: las carpetas __pycache__/ y .pytest_cache/ (son temporales; van al .gitignore).

Importante: subir los archivos de test NO hace que se ejecuten solos en GitHub. Para que corran automáticamente en cada push y aparezca el ✓/X (eso es **CI/CD**), hay que agregar un archivo en .github/workflows/ con **GitHub Actions**. Es un paso opcional siguiente.

Resumen para arrancar hoy: (1) completar la tabla de casos de prueba manuales; (2) correr pytest y ver los verdes; (3) más adelante, sumar pruebas de API y, si se quiere, CI/CD con GitHub Actions.